

Register Windowing and Renaming

- The reduced hardware requirements of RISC processors leave additional space available on the chip for the system designer. RISC CPUs generally use this space to include a large number of registers (> 100 occasionally).
- The CPU can access data in registers more quickly than data in memory so having more registers makes more data available faster. Having more registers also helps reduce the number of memory references especially when calling and returning from subroutines.

Register Windowing and Renaming

- The RISC processor may not be able to access all the registers it has at any given time provided that it has many of it.
- Most RISC CPUs have some global registers which are always accessible. The remaining registers are windowed so that only a subset of the registers are accessible at any specific time.

Register Windows

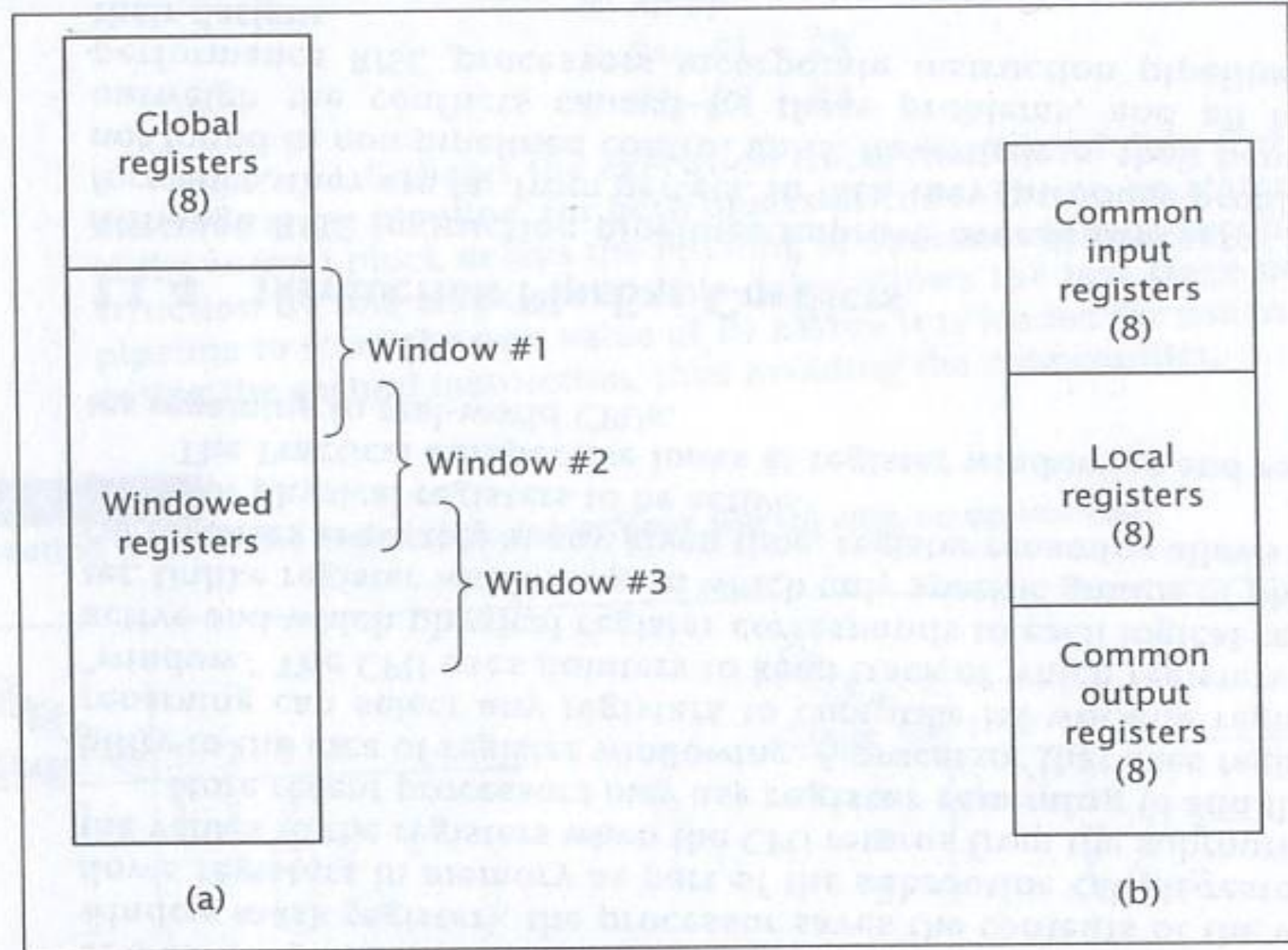
- To understand how register windows work, we consider the windowing scheme used by the Sun SPARC processor.
- The processor can access any of the 32 different registers at a given time. (The instruction formats for SPARC always use 5 bits to select a source/destination register which can take any 32 different values.
- Of these 32 registers, 8 are global registers that are always accessible. The remaining 24 registers are contained in the register window.

Register Window

- On figure 11.4, the register window overlap. The overlap consists of 8 registers in SPARC CPU. Notice that the organization of the windows are supposed to be circular and not linear; meaning that the last window overlaps with the first window.
- Example: the last 8 registers of window 1 are also the first 8 registers of window 2. Similarly, the last 8 registers of window 2 are also the first 8 registers of window 3. The middle 8 registers of window 2 are local; they are not shared with any other window.

FIGURE 11.4

Register windowing in the SPARC processor



Register Window

- The RISC CPU must keep track of which window is active and which windows contain valid data. A window pointer register contains the value of the window that is currently active. A window mask register contains 1 bit per window and denotes which windows contain valid data.

Register Window

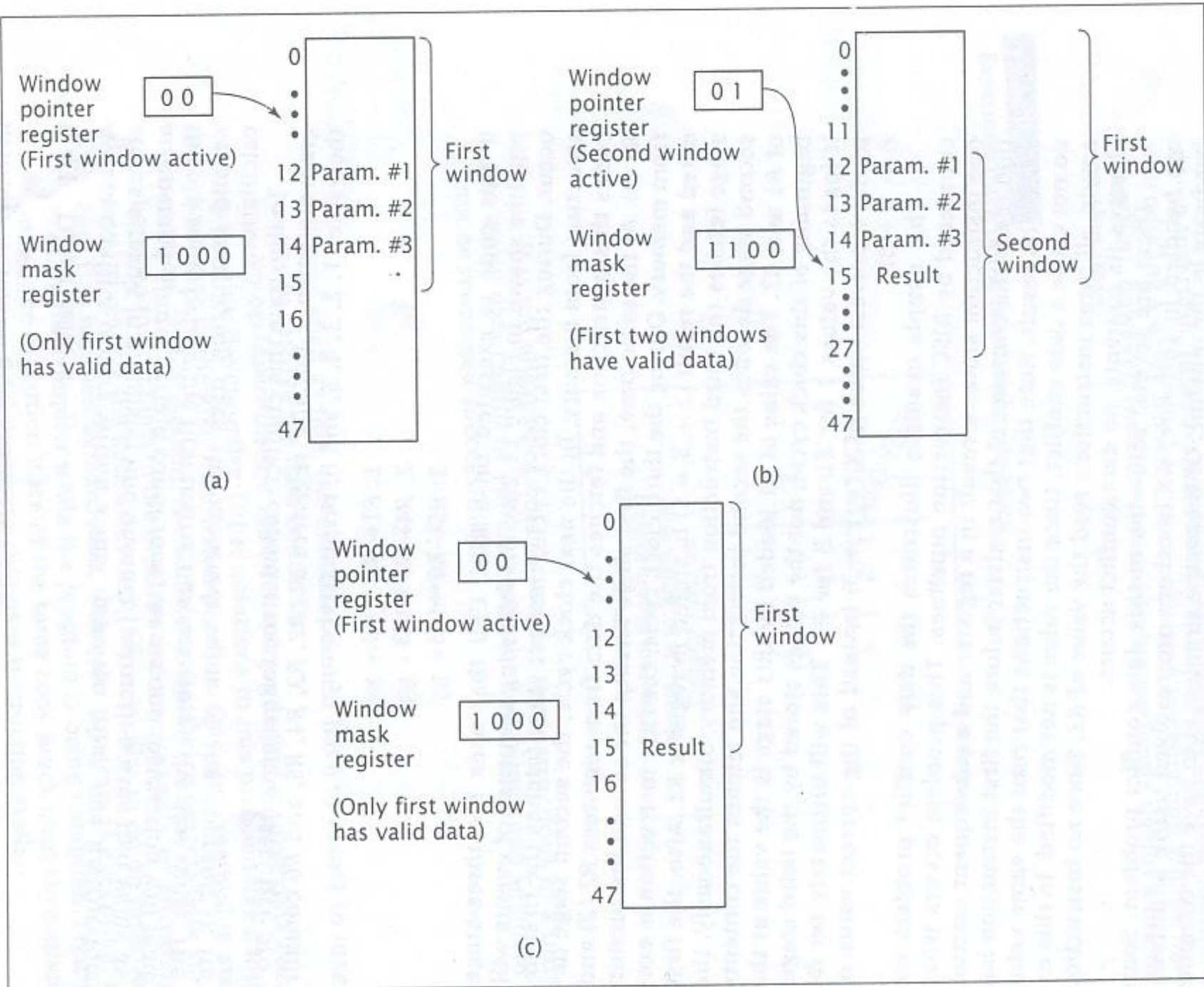
- Register windows provide their greatest benefit when the CPU calls a subroutine. During the **calling process**, the **register window is moved down 1 window position**. In the **SPARC CPU**, if **window 1 is active** and the **CPU calls a subroutine**, the processor **activates window 2 by updating the window pointer and window mask registers**. The CPU can **pass parameters to the subroutine via the registers that overlap both windows** instead of memory. This saves a lot of time when accessing data. The CPU can use the same registers to return results to the calling routine.

Register Window Example

- Example: A CPU with 48 registers. The CPU has 4 windows with 16 registers each and an overlap of 4 registers between windows. Initially the CPU is running a program using the 1st register window (shown in 11.5a). It must call a subroutine and pass 3 parameters to the subroutine. The CPU stores these parameters in 3 of the 4 overlapping registers and calls the subroutine. Figure 11.5b shows the state of the CPU while the subroutine is active. The subroutine can directly access the parameters. Now the subroutine calculates a result to be returned to the main program. The CPU deactivates the 2nd window; the CPU now works with the 1st window as shown in figure 11.5c and can directly access the result.

Figure 11.5

Register windowing in a CPU: (a) during execution of the main routine, (b) executing a subroutine, and (c) after returning from the subroutine



Register Windows

- Most RISC processors that use register windowing have about 8 windows which is sufficient for the vast majority of programs to handle subroutine calls. Besides recursive subroutines, it is unusual for a program to go beyond 8 levels of calls. However it is possible for the **level of calls to exceed the number of windows** and the CPU must be able to handle this situation.

Register Renaming

- Unlike **register windowing**, in which **only specific groups of physical registers are active at any given time**, **register renaming allows any group of physical registers to be active**.
- The Practical Perspective looks at register windowing and register renaming in real-world CPUs.

Register Renaming

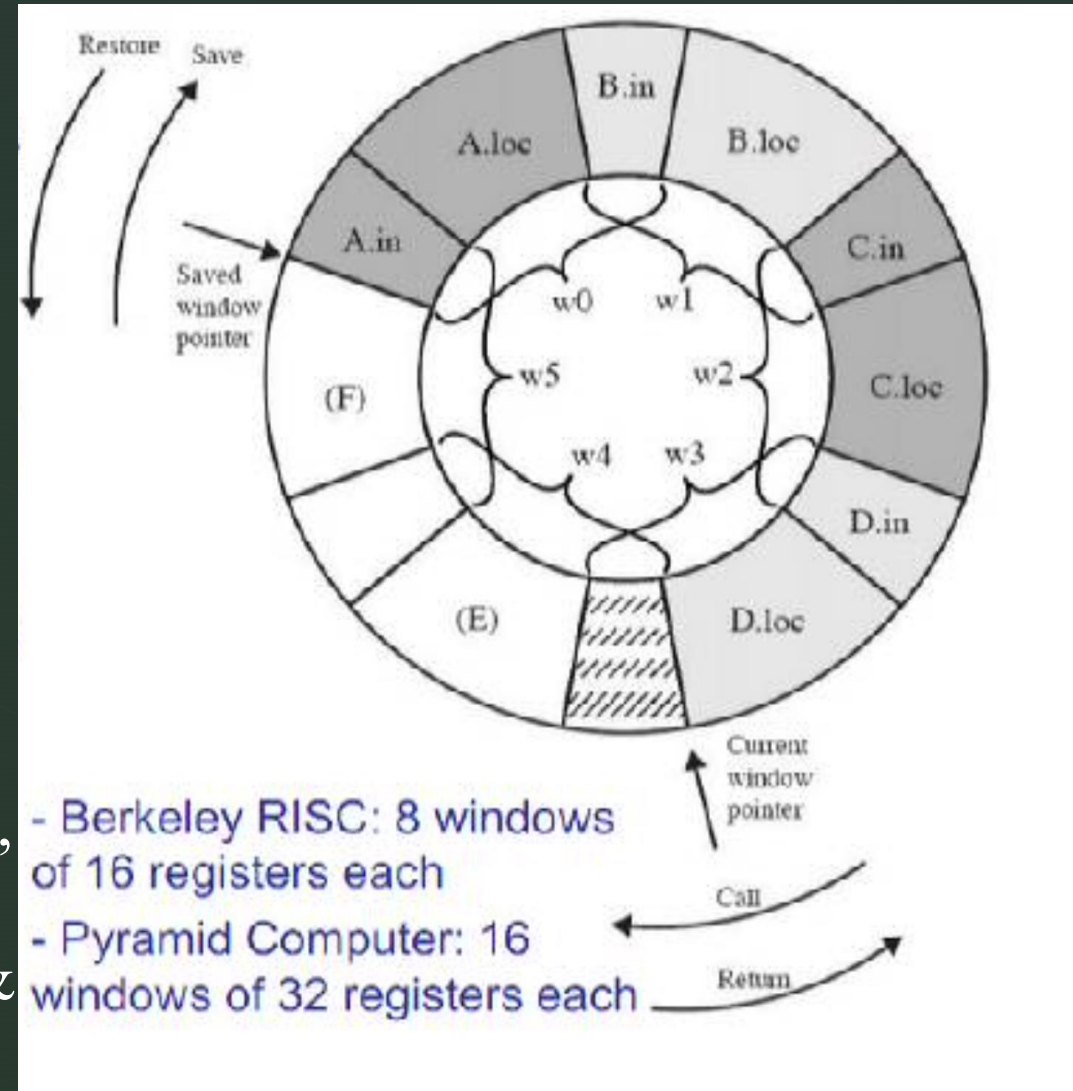
- More recent processors may use register renaming to add flexibility to the idea of register windowing. A processor that uses register renaming can select any registers to create its working register “window.” The CPU uses pointers to keep track of which registers are active and which physical register corresponds to each logical register.

Practical Perspective

- Both register windowing and register renaming are used in modern processors. In addition to SPARC processors, the RISC II processor uses **register windowing**. The **RISC II** processor has **138 registers**; 10 are global and the rest are windowed. Each of its 8 windows contain 22 registers: 6 common input, 10 local, and 6 common output. The UltraSPARC III processor also uses register windowing. The MIPS R10000 on the other hand uses register renaming.
- Intel's Itanium processor takes a unique approach. It has **128 integer registers**, 32 of which are global. The remaining 96 registers are windowed, but the size of the window is not fixed; their size can be varied depending on the needs of the program under execution.

Circular buffer organization of overlapped windows

- A circular buffer of 6 windows. The buffer is filled to a depth of 4 (A called B, B called C, C called D) with procedure D active.
- CWP points to the window of the currently active procedure.
- Reg references are offset by CWP to determine the actual physical reg.
- SWP identifies the window most recently saved in memory.
- If procedure D now calls procedure E, arguments for E are placed in D's temporary regs (overlap between w3 & w4), and CWP is advanced by one window.



Example:

The system has **74** registers:

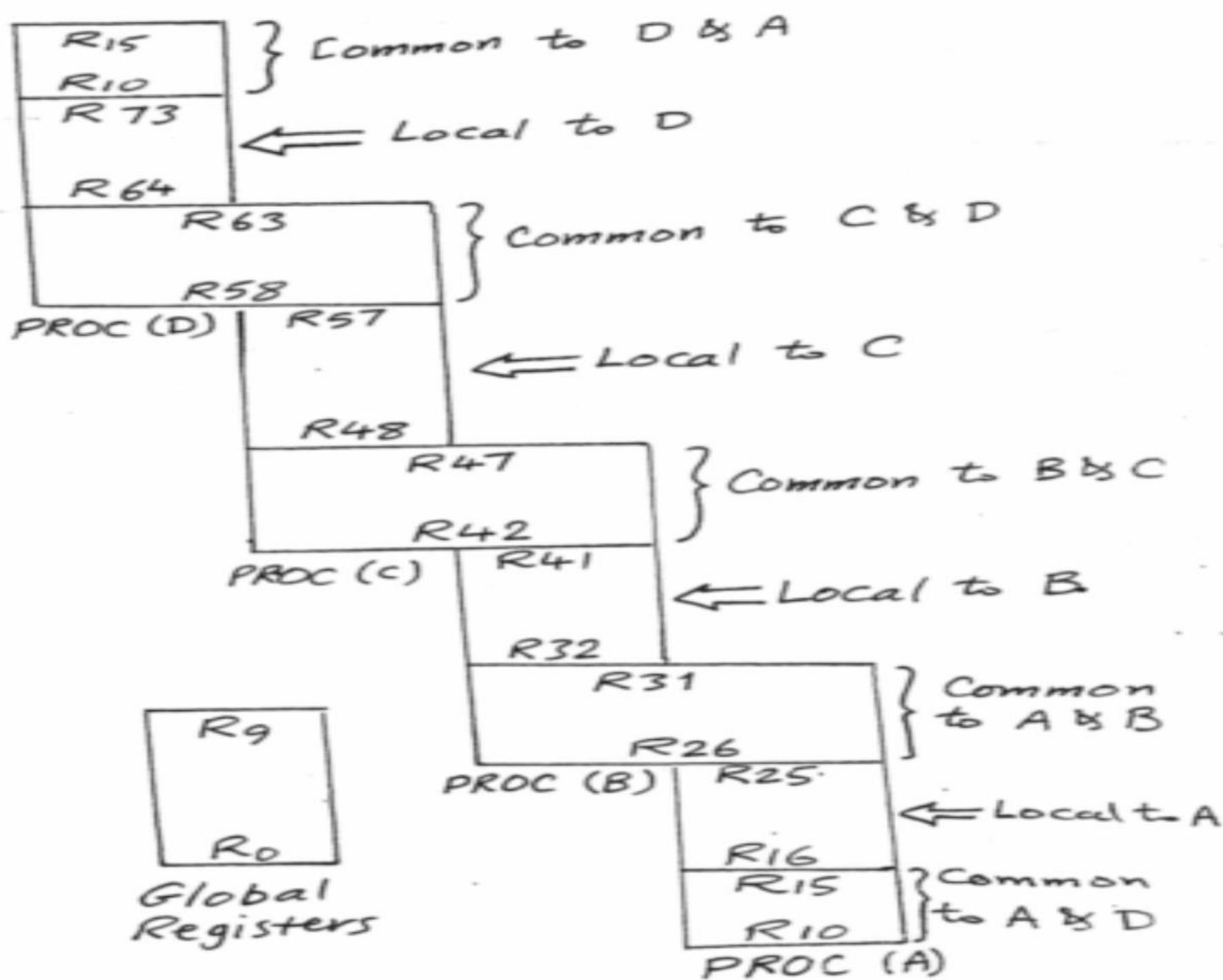
R₀-R₉: Ten hold parameters shared by all procedures.

R₁₀-R₇₃: Sixty four registers are divided into FOUR windows to accommodate procedures A, B, C & D.

- Each register window consists of **10 local registers** and **TWO sets of 6 registers common** to adjacent windows.
- **Local Registers**: used for local variables.
- **Common Registers**: used for exchange of parameters and results between adjacent procedures.
- Each procedure has 32 registers when it is active, these are;

global regs + local regs + overlapping regs

10+10+6+6= 32 Registers



Register Window Size:

G: No. of global registers.

L: No. of local registers in each window.

C: No. of registers common to two windows

W: No. of windows.

- **Window size (S):** No. of registers available for each window.

$$\text{Window Size} = S = L + 2C + G$$

- **Register file (F):** Total no. of registers needed in the processor.

$$\text{Register File} = F = (L + C)W + G$$

For the given example:

G=10, L=10, C=6 and W=4, then

$S = L + 2C + G = 32$ regs, and $F = (L + C)W + G = 74$ regs